Parallel Numerical Library

Complete project documentation, for my own reference

Olajide Badejo

August 2026

Contents

Chapter 1

Project overview

1.1 What this is

A C++23 numerical methods library with a pluggable parallel execution backend layer. Twelve iterative linear solvers are implemented exactly once, against a single problem interface, and run unchanged over seven execution backends: serial, OpenMP, POSIX threads, a `std::jthread` pool, MPI, hybrid MPI with OpenMP threads, and CUDA.

Around that core sit a numerics module (root finding, quadrature, ODE integration, dense LU and QR), two problem families (the 2D Poisson five point stencil and seeded dense systems), a benchmark harness, and three reports.

1.2 Why I built it this way

The project merges two earlier ideas, a numerical methods library and a parallel scientific computing study, and the merge is only worth doing if it produces something neither would alone. The thing it produces is this: **one numerical library, many execution backends**, so that a benchmark can isolate what each programming model costs on identical numerical work.

The usual comparison between parallel programming models is compromised before it starts, because each model gets its own implementation and the study ends up measuring the care that went into each one. Here a solver does not know which backend it is on and a backend does not know which solver it is running, so the only variable between two measurements is the execution model.

That only works if "identical numerical work" is literally true, which is why the central engineering commitment of the project is bit identical results across backends, verified by test rather than asserted.

1.3 Goals

1. A numerical core where every result carries its own evidence: value, error estimate, iteration count, evaluation count, converged flag and an explicit stop reason. No caller can consume an unconverged answer by accident.

2. A solver zoo presented as one family tree, with each method derived from the common splitting framework and each convergence claim checked against closed form theory rather than asserted.

3. Execution backends behind one interface, with identical numerics as a tested invariant.
4. A backend cost study: the same solver on the same seeded problem across every backend, with scaling curves and the heterogeneous core knee.
5. A CPU versus GPU comparison designed so the hardware does not decide the argument.
6. Professional finish: tests, docs, CI, three PDFs, all reproducible from a clean clone with `make all`.

1.4 Scope, and what is deliberately outside it

Inside: iterative linear solvers on two problem families, seven execution models, bandwidth bound performance analysis, and convergence theory for the model problem.

Outside, and deliberately so:

- **Sparse matrix formats.** The stencil is matrix free and the dense problem is dense. A general sparse format would add a large amount of code that the questions being asked do not need.
- **Preconditioned conjugate gradient.** The pieces exist and it is listed under further work, but it was not required and the project was already large.
- **Multigrid.** Same reasoning. It would change the iteration count story entirely and deserves its own project.
- **Compute bound kernels.** Every device conclusion here is about bandwidth bound stencil work, and I say so wherever a conclusion is stated.
- **Power, cost, and programmer time.** Not measured, not claimed.

Chapter 2

Specifications

2.1 Target machine

Component	Specification
CPU	Intel Core i7-14700K, 8 performance cores plus 12 efficiency cores, 28 threads
RAM	31.7 GB host, 12 GB budgeted to the WSL2 guest
GPU	NVIDIA GeForce RTX 5070, 12227 MiB, compute capability 12.0, 48 SMs
Driver	610.62
OS	Windows 11 Pro 10.0.26200, everything running inside WSL2
Guest	Ubuntu 26.04 LTS, kernel 6.18.33.2-microsoft-standard-WSL2

2.2 Toolchain as actually installed

Every version below was verified by running the tool, not by reading a package list. Where the installed version differs from what the specification asked for, the substitution and its reason are recorded.

Component	Specification floor	Installed
Host compiler	GCC 16.1	GCC 16.0.1 20260322 (experimental)
C++ standard	C++23	`__cplusplus` 202302
CMake	4.4	4.4.0 via pipx
MPI	OpenMPI 5.0.x	OpenMPI 5.0.10
OpenMP	6.0 cited	`_OPENMP` 202111, which is 5.2
CUDA	13.3	13.3, V13.3.73
CUDA host compiler	not specified	GCC 14
Python	3 with matplotlib, pandas	3.14.4, matplotlib 3.10.7, pandas 2.3.3
LaTeX	TeX Live with latexmk	latexmk 4.87
Build tool	not specified	Ninja 1.13.2
Lint	ruff	0.15.22
Format	clang-format	20.1.7

2.2.1 Why GCC 16 over the stable GCC 15

GCC 16.0.1 is an experimental trunk snapshot and GCC 15.2.0 is a stable release, so the safe choice looked like 15. I tested both against the features the project actually needs and chose 16 on merit rather than on version number:

- GCC 16 provides `<mdspan>`, which GCC 15 does not. That is directly useful for a 2D grid.
- GCC 16 reports `_OPENMP` 202111, which is OpenMP 5.2. GCC 15 reports 201511, which is 4.5.
- Both compile the full C++23 feature set the project uses: `jthread`, `barrier`, `expected`, ranges, `print`.

2.2.2 Deliberate deviations from the specification

WSL processor count. The specification asks for `processors=20` in `.wslconfig`. The file already set `processors=28`, exposing every host logical processor, which is a superset of what a sweep to twenty workers needs. It also carries a dated comment explaining that a lower count makes the hybrid core comparison impossible to measure, and documents a machine crash that the memory budget in the same file exists to prevent. I left it alone.

OpenMP level. The 6.0 specification is cited as the reference document, as asked, but GCC implements to 5.2 and the code is restricted to that. The build asserts `_OPENMP >= 201511` so a downgraded toolchain fails loudly rather than silently producing different code.

2.3 Constraints that shaped the design

1. **No em dashes or en dashes anywhere, in any file type, including compiled PDFs.** Enforced by a linter from the first commit. This also rules out the LaTeX `--` and `---` ligatures in prose, and BibTeX page ranges, both of which typeset as the banned characters.
2. **Reproducibility is a deliverable.** A clone plus `make setup && make all` reproduces every number. Seeds recorded, nothing hand copied.
3. **No number without a run.** Pending values read `pending`. Every result row carries backend, worker count, pinning, seed and commit hash.
4. **Honest comparison.** The report never headlines a GPU speedup without the device normalised framing beside it.
5. **12 GB memory ceiling in the guest.** This bounds problem sizes and is why build parallelism is capped at six jobs.

Chapter 3

Implementation details

3.1 Core types and diagnostics

`include/pnl/core/` holds the vocabulary. `Real` is `double` throughout; there is no mixed precision path, so a single alias keeps the CUDA boundary unambiguous. `Index` is `std::ptrdiff_t`, deliberately signed: OpenMP canonical loop form accepts unsigned indices only from 3.0 onward, several compilers still generate worse code for them, and a backward sweep is expressible without wraparound traps.

`Diagnostics` is the record attached to every numerical result: error estimate, iterations, evaluations, converged flag, and a `StopReason` enumeration distinguishing convergence from hitting the cap, stagnation, breakdown and divergence. Hitting an iteration cap is never indistinguishable from meeting a tolerance.

`block_partition(n, parts, k)` lives here rather than inside a backend, because both the chunk grid and the MPI row decomposition use it. It is remainder aware: the first `n mod parts` blocks get one extra element, so sizes differ by at most one and nothing is dropped.

3.2 The backend interface, and the decision it rests on

`backend.hpp` declares `parallel_for`, `reduce`, `barrier`, plus `local_rows`, `exchange_halo`, `gather_rows` and `run_ordered` for the distributed case. It is the minimal common denominator of the models it spans.

3.2.1 Chunk level callbacks

The body of a `parallel_for` receives a `Range` and loops over it itself, rather than being called once per element. This was the first significant design decision and it has two consequences I wanted:

- The `std::function` indirection is paid once per chunk, so it is proportional to the worker count and not to the problem size. It does not contaminate the timings.
- The innermost loop stays a plain loop over contiguous memory that the compiler can vectorise, so each backend measures its own dispatch cost rather than a penalty the abstraction imposed on everything equally.

3.2.2 Determinism, which turned out to be the keystone

Floating point addition is not associative, so a reduction's answer depends on the grouping of its partial sums. Most libraries treat this as unavoidable. I made it a property of the design instead.

Under `ReductionMode::Deterministic` the chunk grid is fixed by the problem size alone, `min(512, n)` chunks, never by the worker count. Each chunk's partial goes into its own slot and the slots are summed in ascending index order. Which worker computed which slot, and when it finished, cannot affect the result.

The payoff is larger than I expected when I wrote it. Because every shared memory backend produces bit identical iterates at every worker count, the equivalence suite can assert *exact equality* rather than a tolerance. That makes it a far more sensitive instrument: it fails on the first bit that differs. The fused multiply add discrepancy described later was one bit in the last place and any reasonable tolerance would have swallowed it.

3.3 Problems

3.3.1 Poisson2D

The five point stencil on the unit square. Stored as an $(n + 2)$ by $(n + 2)$ row major grid with a one cell boundary ring holding the homogeneous Dirichlet values, so there is no boundary branch in the innermost loop and the ring doubles as the MPI halo. The scaling by h^2 makes the matrix have diagonal 4 and off diagonals -1.

`PoissonTheory` computes the closed form spectral quantities so that no test re-derives them: $\rho_J = \cos(\pi h)$, $\rho_{GS} = \rho_J^2$, $\omega^* = 2/(1 + \sin(\pi h))$.

Two right hand sides exist and the reason is important enough that it has its own section under problems below.

3.3.2 DenseProblem

Seeded dense systems in two flavours: strictly diagonally dominant (guarantees Jacobi and Gauss Seidel converge, but not symmetric so conjugate gradient does not apply) and symmetric with a dominant positive diagonal (positive definite by Gershgorin, so admissible for conjugate gradient and for the Ostrowski and Reich theorem).

The exact solution is chosen first and the right hand side formed from it, so the unit tests compare against something that is not another solver's output. Diagonal blocks are LU factorised once in the constructor and reused every sweep, which is what makes the block methods competitive rather than merely correct.

3.4 The solver zoo

Twelve solvers, eleven of which are one splitting family. Each lives in its own header, documents its splitting, its convergence order and the exceptions it throws, and hands a sweep to a shared driver. The shared driver is what guarantees every method measures its residual the same way, stops on the same criterion, and pays the same reporting overhead.

The block methods take a block count as a *solver parameter and never the worker count*, which is what keeps their iterates independent of how many workers happen to be running. Each problem defines its natural block: a grid line solved by the Thomas algorithm for the stencil, contiguous index ranges solved by dense LU for the dense problem.

3.5 What each backend does underneath

- **serial** walks the same chunk grid as the parallel backends rather than looping over the whole range, so "serial equals parallel bit for bit" is a statement about the parallel backends and not an artefact.
- **openmp** is the only backend that does not partition the range itself; it hands the chunk grid to OpenMP's worksharing construct, so what is measured is OpenMP's scheduler.
- **pthreads** uses raw POSIX primitives and a persistent pool. Workers wait on a *generation counter* rather than a flag, which removes the lost wakeup race a boolean would have.
- **jthread** uses `std::jthread`, `std::stop_token` and two `std::barrier` phases in strict alternation. Deliberately no work stealing; the schedule cost measurement is the evidence.
- **mpi** uses remainder aware row decomposition and two `MPI_Sendrecv` calls per halo exchange. Each rank allocates the whole grid and computes its own band, which spends memory to keep the solver code identical across backends; the communication volume is unaffected.
- **hybrid** inherits from the MPI backend rather than reimplementing, so any difference the sweep measures between them is the threading and nothing else.
- **cuda** is a separate device solve path, not a `Backend`. The state stays in device memory for the whole solve; forcing it behind `parallel_for` would measure PCIe latency.

3.6 Instrumentation

The sweep harness resolves `benchmarks/sweep_matrix.yaml`, which declares nine blocks each carrying a `why` field saying what question it answers. The driver is resumable, records provenance on every row, merges atomically through a temporary file and rename, and writes a session manifest containing both devices' bandwidth probes and the toolchain versions.

Figures and tables are generated from the summary by `scripts/gen_report_assets.py`, which is idempotent: the only inputs are the summary and the manifest, and every output is overwritten in full.

Chapter 4

Chronological account

4.1 Phase 0: environment

Started by probing rather than assuming. Found the hardware matched the specification exactly, and that a `.wslconfig` already existed with a documented crash history, which I left alone.

Found GCC 16 available in apt and tested it against GCC 15 on the features the project needs, choosing 16 for `<mdspan>` and OpenMP 5.2. Installed CMake 4.4 through pipx since the distribution ships 4.2.3.

Probed MPI and found the version macro says 3.1 while MPI 4.0 entry points link. Probed CUDA and hit the first structural problem: nvcc rejects GCC 16, and cannot parse GCC 15's headers either. Resolved with a C ABI boundary.

Spent time on a false lead here: the CUDA configure failure looked like it might be caused by spaces in the project path. A controlled test disproved it, and the same test also confirmed that CMake, CUDA and latexmk all work in a path with spaces, which was worth knowing.

Wrote the dash linter first, because it gates everything. Its first run found an em dash in the build specification itself.

4.2 Phase 1: numerics and serial solvers

Wrote the core types, the backend interface, the Poisson problem, and validated early with a smoke test before building further. That smoke test paid for itself immediately: it confirmed the Jacobi contraction factor matched $\cos(\pi h)$ to six digits, the Jacobi to Gauss Seidel ratio was exactly 2.0, and the discretisation error constant was 0.823 against the predicted $\pi^2/12 = 0.8225$. Knowing the foundation was right made everything after it cheaper to debug.

Then the numerics module, the dense problem, the solver zoo, and the test harness.

4.3 Phase 2: convergence tests

Wrote the tests that check measurements against closed form theory. Three faults surfaced here, all in solvers rather than tests: Richardson diverging from a bad default, Richardson diverging from a bad step estimator, and the conjugate gradient count refusing to grow with the grid. The last of these turned out to be the eigenvector problem and was the most interesting bug of the project.

4.4 Phase 3: shared memory backends

OpenMP, pthreads and the jthread pool, then the equivalence suite. The equivalence suite passed on its first run, which I did not expect; the determinism design carried it. Found the jthread destructor ordering hazard by inspection while reviewing the shutdown path.

4.5 Phase 4: distributed backends

MPI and hybrid. Four test cases failed at once at two and four ranks, all from one cause: the returned solution was left distributed while the tests compared the whole vector. Adding a single end of solve gather fixed all four. While reviewing the dense path I found a second, latent fault: block ranges and row ranges differ, and a gather assuming otherwise would have corrupted vectors silently.

4.6 Phase 5: CUDA

The link broke in a way that took two wrong guesses to diagnose. Then duplicate device stubs from kernels in a header. Then a host pointer passed to a device to device copy. Then the one bit fused multiply add discrepancy, which was the only one of the four that taught me something about the project rather than about the tools.

Confirmed the GPU sweeps are bit identical to the CPU, the red black iteration penalty is 2.6 percent, and the device triad bandwidth is around 548 GiB/s.

4.7 Phase 6: the sweep

Declared the matrix, then found two faults in my own declaration before running it: two quadratic methods listed in a block meant for linear ones, and the resume logic testing on the wrong side of the work. Both were caught by thinking about the arithmetic rather than by waiting for the consequences.

4.8 Phases 7 to 9

Documentation written from the measured numbers, the three reports, and final quality assurance.

Chapter 5

Problems and errors

The full record is in `docs/ENGINEERING_LOG.md` and in the debug report, with symptom, root cause, options, fix and verification for each. This chapter records what I want to remember about them rather than repeating them.

5.1 The five that produced plausible numbers

These are the ones worth internalising. A crash teaches nothing; a wrong answer that looks right is the failure mode that survives.

The manufactured solution was an eigenvector. With $u = \sin(\pi x)\sin(\pi y)$, conjugate gradient converged in one iteration at every grid size, because the source is exactly the lowest eigenvector of the stencil and the Krylov subspace is therefore one dimensional. It looked like a spectacular result.

What caught it was a test asserting the iteration count should *grow* with the grid, not merely that the solver converged. I would not have written that test if the specification had not asked for growth orders to be verified.

The fix needed thought, because the obvious alternatives are worse: a polynomial manufactured solution has vanishing fourth derivatives, which makes the stencil exact and destroys the second order accuracy test; a sum of a few modes just moves the problem. Keeping both right hand sides and using each where it is honest was the right answer, and the single mode source is genuinely *ideal* for the stationary methods, because the slowest Jacobi mode is that same lowest mode.

A default that suited the first solver written. `relaxation` defaulted to 1.0, harmless for SOR and catastrophic for Richardson. The generalisation: a shared options struct wants defaults that mean "ask the component", not defaults that happen to suit one of them.

The resumable sweep was not resumable. The skip test sat after the work. Counters reported plausible numbers throughout. It survived until I timed a restart instead of reading its output.

A tolerance below the arithmetic. Two tests asked for 10^{-13} from stationary iterations whose rounding floor on this operator is around 3×10^{-13}. The failure appeared only when the solver those tests used changed, so it looked like a regression in the solver. Measuring the floor turned a confusing symptom into a table.

One bit of fused multiply add. The host was contracting $ab + cd$ and the device was not. Visible only because the equivalence suite asserts bit identity. Following it led to a real weakness in the project's central claim, and the fix, disabling contraction on both sides, made the claim robust to compiler upgrades as well.

5.2 The toolchain problems

Four of these, all in phase 0 or phase 5, all structural: nvcc rejecting the project compiler, a CMake setting arriving after `project()`, MPI version macros describing conformance rather than capability, and the CUDA host compiler's library directory hijacking the link.

The last one cost the most time and taught the most: I guessed twice before diagnosing it, first blaming spaces in the path and then blaming the linker driver. The lesson is that when a link fails on symbols from a component that has nothing to do with the change, the change probably altered the *search order* rather than the code.

5.3 The one the environment imposed

WSL2 hides the hybrid core topology. I could not identify performance versus efficiency cores from inside the guest, and thread affinity binds to a virtual processor the hypervisor may place anywhere.

The response I am happiest with in the whole project: rather than assume the conventional processor enumeration and produce confident labels with nothing behind them, the library measures, reports what it could establish, and takes the knee from the aggregate scaling curve, which does not depend on identifying individual cores. The claim got weaker and became true.

Chapter 6

Future work

6.1 Preconditioned conjugate gradient

The most obvious gap. Symmetric Gauss Seidel and SSOR are already implemented and are exactly the preconditioners the method wants; they currently exist in the zoo without the application that motivates them. This is a small amount of code and would substantially improve the strongest solver in the library.

6.2 Multigrid

Every component is present: smoothers in the relaxation sweeps, an exact coarse solve in the LU routine, the grid hierarchy implicit in the Poisson problem. Multigrid makes the iteration count independent of the grid, which would put the $O(n^2)$ and $O(n)$ results of this project in their proper context as the two things it improves on. It deserves its own project.

6.3 A local slab MPI decomposition

Each rank currently allocates the whole grid. The communication volume is already correct so no timing here would change, but it would remove the memory ceiling that keeps distributed runs below the sizes the device comparison uses.

6.4 Bare metal measurement of the core knee

The one objective the environment prevented. Running the same sweep outside a hypervisor would settle whether the knee in the aggregate curve coincides with the performance core count, which is currently inferred rather than confirmed.

6.5 Mixed precision

The stencil sweeps are bandwidth bound, so halving the width of the iterate would come close to halving the time. Whether the convergence penalty is worth it is exactly the kind of question this library's separation of iteration count from per iteration cost is built to answer, and it would also test whether the determinism design survives a second scalar type.

6.6 Smaller things worth doing

- **An allocation free block Jacobi.** It currently snapshots the previous iterate each sweep, which is an $O(N)$ copy that could be avoided by double buffering.
- **Overlapping halo exchange with interior computation.** The standard optimisation, and the library is structured to make it expressible: compute the interior while the halo is in flight, then the boundary rows. It would improve MPI scaling and the measurement infrastructure to prove it already exists.
- **Persistent MPI collectives.** The probe confirmed `MPI_Allreduce_init` links even though the version macro says 3.1. Worth measuring whether it helps the conjugate gradient reduction cost.
- **A device resident multi solve mode.** Transfer time is currently reported separately and honestly, but a mode that keeps state resident across solves would show what a real application would see.